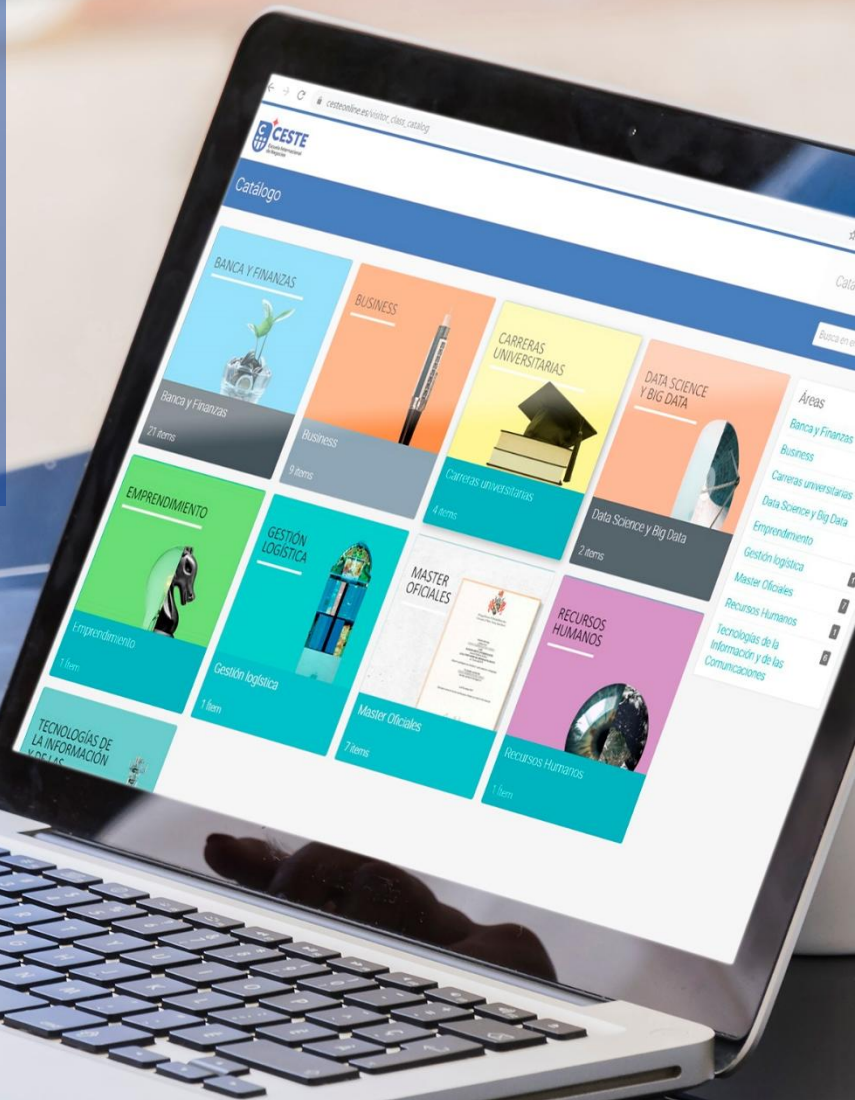


CESTE

Escuela Internacional de Negocios

Zaragoza (España)



¿Quién soy?



- Pablo Sanz Caperote
- Doble Grado en Ingeniería Informática – Matemáticas
- Profesional con más de 3 años de experiencia entorno al mundo de los datos.
- Casi 2 años trabajando con Databricks
- Varias certificaciones en Clouds (AWS, Azure, GCP)



www.linkedin.com/in/pablosanzcaperote

Optimización en Spark



Índice del curso

1. Fundamentos y diagnóstico de rendimiento

- Funcionamiento de Spark
- Cuellos de botella
- Uso de herramientas

2. Técnicas de optimización

- Buenas prácticas con código
- Optimización de particionado y archivos
- Uso de Delta Lake en Databricks

3. Diseño de pipelines escalables y sostenibles

- Modularización, parámetros y manejo de errores
- Orquestación con Databricks Workflows
- Control de costes y clusters eficientes
- Casos reales de optimización de pipelines

1. FUNDAMENTOS Y DIAGNÓSTICO DE RENDIMIENTO

Índice | Fundamentos y diagnóstico de rendimiento

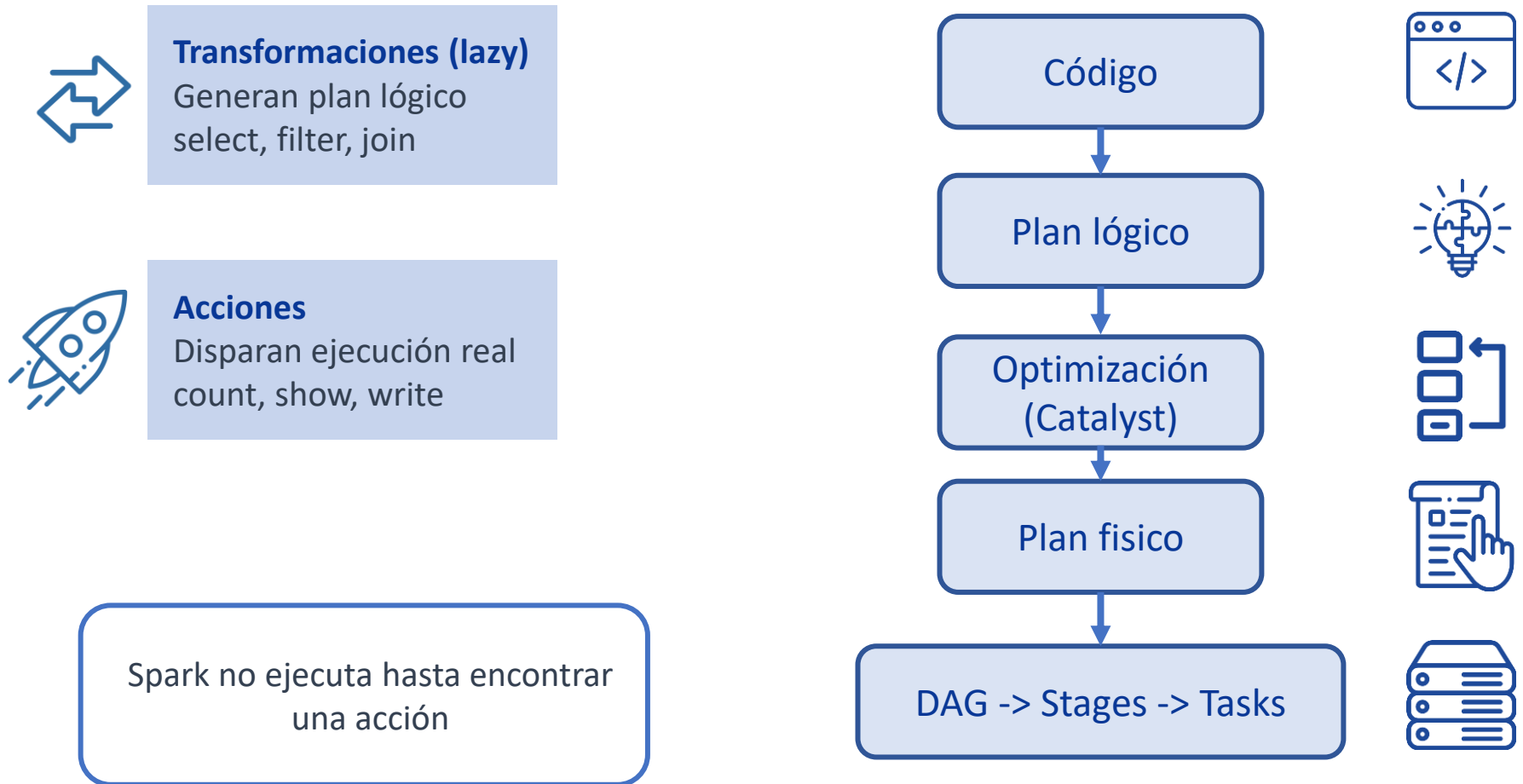
1. Ciclo de ejecución en Spark
2. Arquitectura interna de Spark y Databricks
3. Identificación de cuellos de botella
4. Herramientas de diagnóstico

Objetivos | Conocer las transformaciones de Spark, sus cuellos de botella y las herramientas de diagnóstico

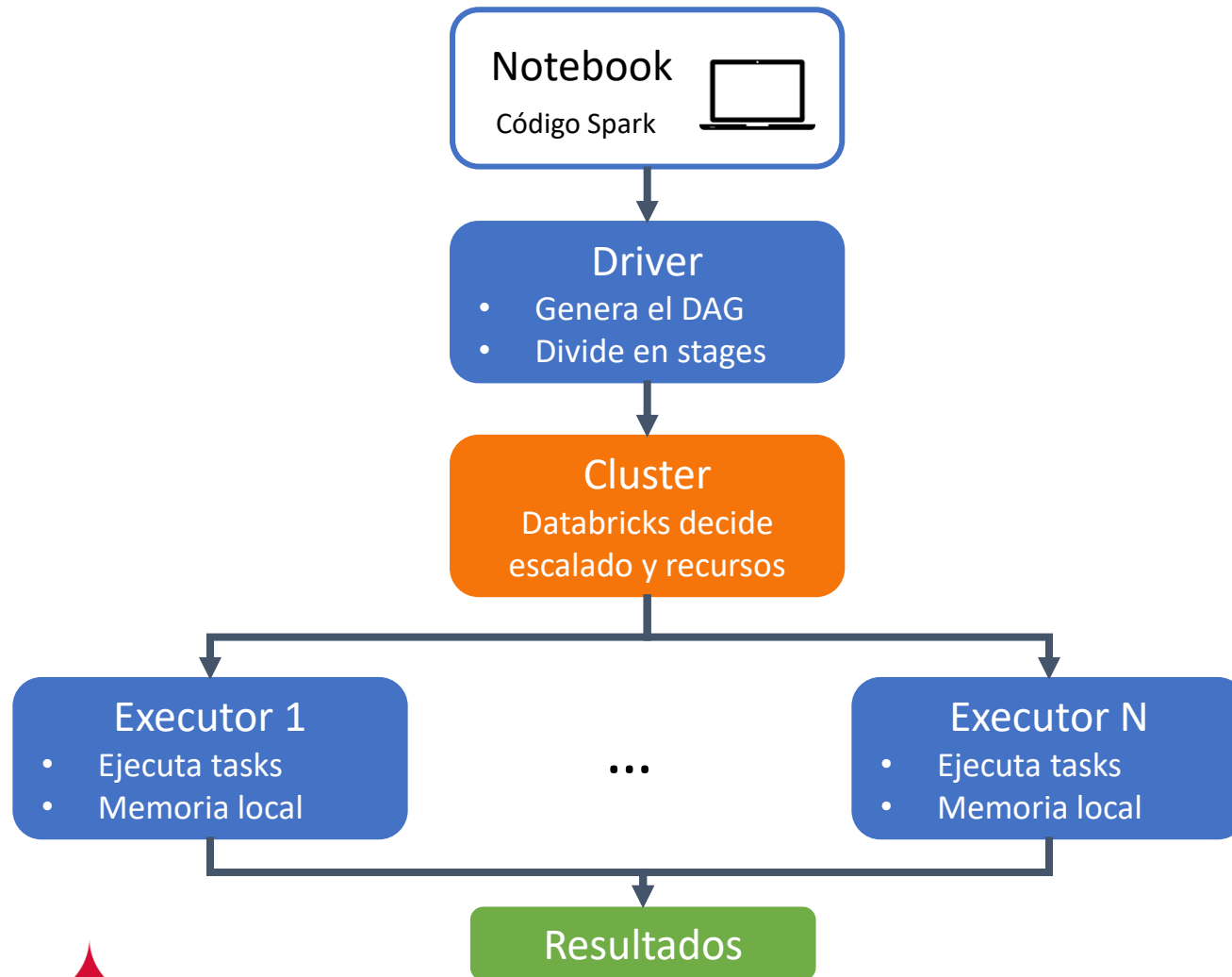


1. Explicar cómo Spark ejecuta un pipeline de datos desde el código hasta la ejecución distribuida.
2. Identificar los cuellos de botella más comunes en Spark.
3. Usar las herramientas de diagnóstico en Databricks

Ciclo de ejecución | Spark planifica primero y ejecuta después



Spark y Databrick por dentro | Spark reparte el trabajo y Databricks gestiona automáticamente



Cuellos de botella | Detectarlos es el primer paso para optimizar un pipeline

Shuffles



- Redistribución de datos entre nodos
- Causas: join, groupBy,...
- Impacto: alto coste

Spark UI

Data Skew



- Particiones con datos muy desbalanceados
- Una tarea tarda mucho más que el resto

DAG Viewer

Particionado ineficiente



- Pocas particiones implica poco paralelismo
- Demasiadas particiones implica sobrecarga de tareas

explain()
queryExecution

DEMO
COMO DIAGNOSTICAR EN SPARK

Concepto	Qué es	Cuándo se crea	Nivel
Job	Acción completa <i>count, save, collect</i>	Se ejecuta una acción	Puede contener varios stages
Stage	Operaciones que pueden ejecutarse sin mover datos	Cada operación que requiere shuffle <i>groupby, join, repartition</i>	Contiene varios task
Task	Unidad indivisible de ejecución	Por partición de los datos	Se ejecutan en paralelo los executors

Ejemplo 1 | Un solo Job, un solo Stage

```
df = spark.range(0, 10)
df.count()
```

- **Jobs:** 1 -> count es acción
- **Stages:** 1 -> No hay shuffle (solo lectura)
- **Tasks:** El número de particiones del DF (el que haya por defecto)

Ejemplo 2 | Un solo Job, varios Stages

```
df = spark.range(0, 10_000).repartition(4)
df.groupBy().sum().collect()
```

- **Jobs:** 1 -> collect es acción
- **Stages:** 2 -> groupby require shuffle
- **Tasks:** 8 -> Cada stage tiene 4 Task

Ejemplo 3 | Varios Jobs

```
df = spark.range(0, 100)

df.count()           # Primer Job
df.filter("id < 50").count() # Segundo Job
```

- **Jobs:** 2 -> cada acción dispara un nuevo Job
- **Stages:** 2 (1 por cada job)
- **Tasks:** El número de particiones del DF (el que haya por defecto)

Ejemplo 4 | Join con shuffle

```
df1 = spark.range(0, 100).withColumnRenamed("id", "key")
df2 = spark.range(50, 150).withColumnRenamed("id", "key")

df1.join(df2, "key").count()
```

- **Jobs:** 1 -> count es acción
- **Stages:** 3 -> join implica shuffle
- **Tasks:** El número de particiones del DF (el que haya por defecto)

KAHOOT



www.ceste.es